



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Calidad de Software

3010440

Albeiro Espinosa Bedoya, Ph.D. Ms.C.
Profesor Asociado

Proceso de Verificación

Agenda (1/1)

1. Proceso de Verificación - ISO 12207
2. Técnicas de Verificación
3. Técnicas Estáticas
4. Técnicas Dinámicas
5. Referencias

1. Proceso de Verificación - ISO 12207

Proceso de Verificación - ISO 12207 (1/5)

ISO/IEC/IEEE 12207 – Procesos del Ciclo de Vida del Software

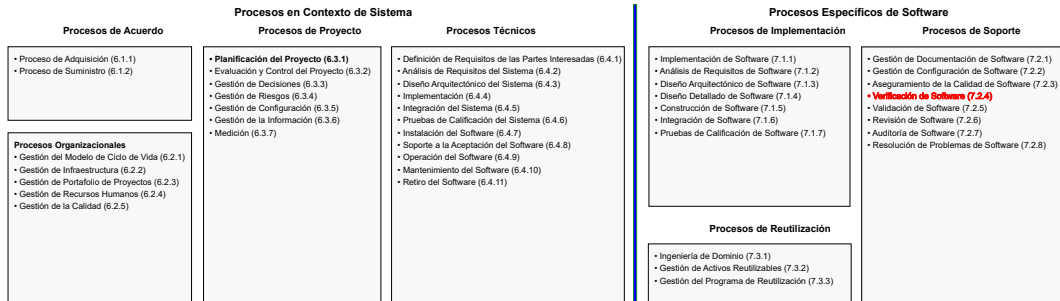


Figura 1: El proceso de Verificación en el Ciclo de Vida.

Proceso de Verificación - ISO 12207 (2/5)

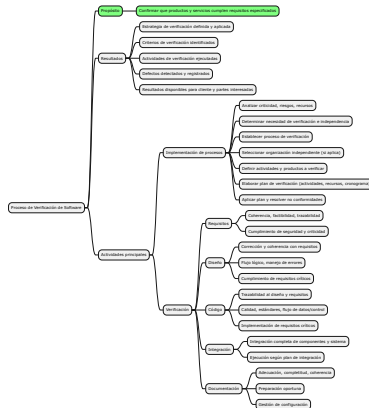


Figura 2: Proceso de Verificación - Propósito

Proceso de Verificación - ISO 12207 (3/5)

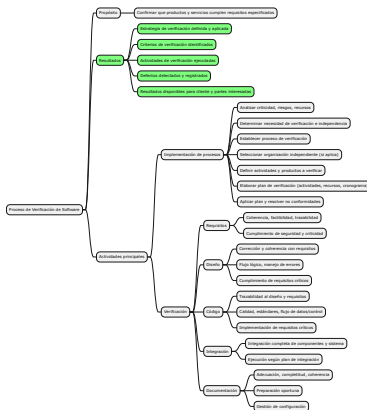


Figura 3: Proceso de Verificación - Resultados

Proceso de Verificación - ISO 12207 (4/5)

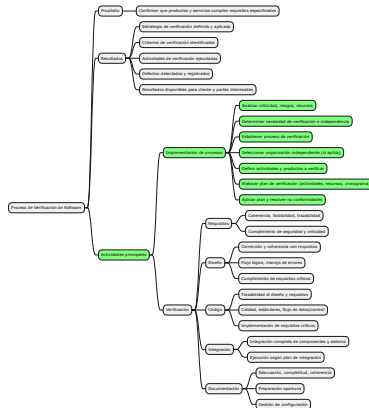


Figura 4: Proceso de Verificación - Resultados - Implementación del Proceso

Proceso de Verificación - ISO 12207 (5/5)

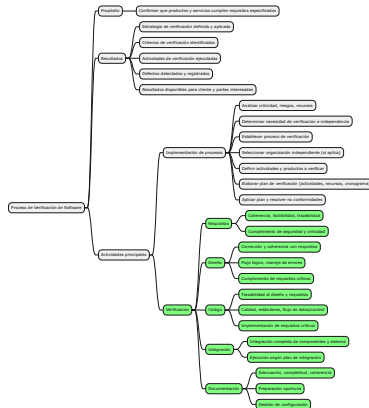


Figura 5: Proceso de Verificación - Artefactos sujetos a Verificación

2. Técnicas de Verificación

Técnicas de Verificación (1/3)

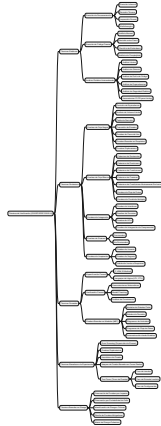


Figura 6: Técnicas de Verificación

Técnicas de Verificación (2/3)

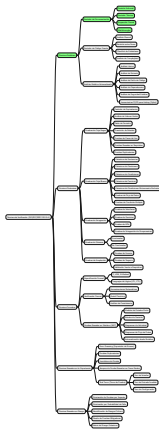


Figura 7: Técnicas de Verificación - Estáticas

Técnicas de Verificación (3/3)

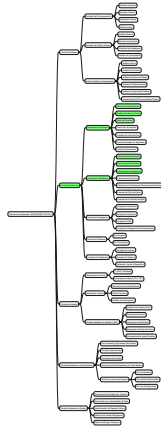


Figura 8: Técnicas de Verificación - Dinámicas

3. Técnicas Estáticas

Técnicas Estáticas

Las técnicas estáticas son métodos de aseguramiento de la calidad que se aplican sin ejecutar el software. Permiten detectar defectos en etapas tempranas del ciclo de vida, revisando artefactos como **requisitos, modelos, código o manuales**, para identificar inconsistencias, ambigüedades u omisiones.

Revisión de Documentación

Consiste en analizar sistemáticamente los artefactos generados durante el desarrollo de software para:

- Verificar cumplimiento de estándares de calidad (claridad, consistencia, completitud).
- Asegurar comprensión por los interesados.
- Detectar errores antes de la implementación o pruebas.

Se aplica a **requisitos, casos de uso, diagramas, código, planes de prueba y manuales.**

Revisión Informal

Método simple y flexible para encontrar errores en documentos.

- **Participantes:** Autor y uno o varios revisores.
- **Estructura:** Sin proceso formal ni roles definidos.
- **Objetivo:** Localizar defectos evidentes, inconsistencias o problemas de claridad.
- **Ejemplos:** Borradores de requisitos, diagramas preliminares.

Ventajas: rápida, económica, adaptable.

Limitaciones: puede omitir defectos sutiles, depende de la experiencia de los revisores.

Revisión Técnica

Introduce mayor formalidad para validar la corrección técnica del documento.

- **Participantes:** Autor, expertos técnicos, arquitectos, testers, usuarios.
- **Estructura:** Reunión planificada con agenda y checklist.
- **Objetivo:** Evaluar viabilidad de implementación y alineación con estándares.
- **Ejemplos:** Validación de modelos de datos, revisión de casos de prueba.

Ventajas: detección temprana de errores de diseño, fomenta la comunicación.

Limitaciones: requiere coordinación y tiempo.

Inspección Formal

Técnica más rigurosa y estructurada, basada en el método de Fagan.

- **Roles:** Moderador, autor, lector, escriba, inspectores.
- **Proceso:** Planificación, preparación individual, reunión de inspección, reparación, seguimiento.
- **Objetivo:** Detectar inconsistencias, omisiones, violaciones de estándares o errores de lógica.

Ventajas: alta tasa de detección de defectos, proceso medible.

Limitaciones: mayor esfuerzo y costo, requiere planificación cuidadosa.

Walkthroughs

El autor guía a los participantes a través del documento, explicando su propósito y decisiones de diseño.

- **Participantes:** Autor, equipo de desarrollo, testers, usuarios clave.
- **Estructura:** Reunión planificada, el autor conduce la sesión y responde preguntas.
- **Objetivo:** Compartir conocimiento, identificar defectos, obtener retroalimentación.
- **Ejemplos:** Presentación de interfaces, revisión de flujos de casos de uso.

Ventajas: aprendizaje colectivo, colaboración.

Limitaciones: requiere moderación para evitar discusiones abiertas.

4. Técnicas Dinámicas

Técnicas Dinámicas

Las técnicas dinámicas requieren la ejecución del programa para detectar defectos. Buscan validar el comportamiento del software en tiempo de ejecución, verificando que cumpla los requisitos funcionales y condiciones de uso. Se clasifican en:

- **Pruebas de Caja Negra:** Basadas en entradas y salidas, sin conocimiento del código.
- **Pruebas de Caja Blanca:** Diseñadas con conocimiento de la estructura interna del software.

Pruebas de Caja Negra

Evalúan el software a partir de entradas, salidas y especificaciones, sin considerar su estructura interna.

- **Objetivo:** Verificar que el sistema responda correctamente a entradas válidas, rechace entradas inválidas y cumpla requisitos.
- **Ventajas:** Independencia del código, participación de usuarios, validación funcional.
- **Limitaciones:** No detecta errores internos, puede requerir muchas pruebas.

Técnicas principales: Partición de Equivalencia, Análisis de Valores Límite, Tabla de Decisión.

Partición de Equivalencia

Divide el dominio de entrada en clases de datos equivalentes para probar un representante de cada clase.

- **Pasos:** Identificar rangos, dividir en clases válidas e inválidas, elegir casos representativos.
- **Ejemplo:** Edad válida 18–65 años: probar 30 (válido), 10 (inválido), 70 (inválido).

Ventaja: reduce casos de prueba manteniendo alta cobertura.

Ejemplo: Partición de Equivalencia

Contexto: Un sistema bancario valida que la edad para abrir una cuenta esté entre 18 y 65 años.

Regla de negocio:

- Válido: $18 \leq edad \leq 65$.
- Inválido: fuera del rango o no numérico.

Paso 1. Identificar el dominio de entrada

- Entrada: valor de la **edad** del cliente.
- Tipo: entero, esperado en el rango $[18, 65]$.

Paso 2. Determinar las clases de equivalencia

Clases válidas:

- C1: Edad dentro del rango válido (18–65).

Clases inválidas:

- C2: Edad menor que 18.
- C3: Edad mayor que 65.
- C4: Valor no numérico.

Paso 3. Seleccionar casos representativos

- C1: edad = 30.
- C2: edad = 16.
- C3: edad = 70.
- C4: edad = “abc”.

Paso 4. Construcción de casos de prueba

Caso	Entrada (edad)	Clase	Resultado esperado
T1	30	C1	Registro aceptado
T2	16	C2	Error: Edad mínima 18 años
T3	70	C3	Error: Edad máxima 65 años
T4	“abc”	C4	Error: Ingrese un valor numérico válido

Análisis de Valores Límite

Se centra en los valores extremos de los rangos de entrada o salida.

- **Pasos:** Identificar límites, probar valores en el límite y cercanos.
- **Ejemplo:** Edad 18-65: probar 18 y 65, además de 17 y 66.

Ventaja: detecta defectos en validaciones de rangos y comparaciones.

Ejemplo: Análisis de Valores Límite

Contexto: El sistema bancario valida que la edad para abrir una cuenta esté entre 18 y 65 años.

Regla de negocio:

- Válido: $18 \leq edad \leq 65$.
- Inválido: fuera del rango o no numérico.

Paso 1. Identificar los límites del dominio

- Mínimo permitido: 18.
- Máximo permitido: 65.

Paso 2. Definir valores críticos

Principio: Los defectos suelen ocurrir en los bordes del rango.

Valores a probar:

- Límite inferior: 17, 18, 19.
- Límite superior: 64, 65, 66.

Paso 3. Casos de prueba representativos

Caso	Entrada (edad)	Descripción	Resultado esperado
T1	17	Debajo del mínimo	Error: Edad mínima 18
T2	18	En el mínimo	Registro aceptado
T3	19	Encima del mínimo	Registro aceptado
T4	64	Debajo del máximo	Registro aceptado
T5	65	En el máximo	Registro aceptado
T6	66	Encima del máximo	Error: Edad máxima 65

Paso 4. Comparación con Partición de Equivalencia

- **Partición de Equivalencia:** basta un valor dentro de cada clase (ej. 30).
- **Análisis de Valores Límite:** se prueban los bordes (17,18,19,64,65,66).
- Se detectan errores comunes en el manejo de límites, como uso de $<$ en lugar de $\leq$.

Paso 5. Representación visual de la Partición



Figura 9: Representación visual de la Partición

Tabla de Decisión

Útil cuando el sistema depende de combinaciones de condiciones lógicas.

- Representa condiciones y acciones en una tabla.
- Cada combinación de condiciones define una regla de prueba.
- **Ejemplo:** Préstamo bancario: historial positivo (Sí/No), ingreso $>$ \$2000 (Sí/No), acciones de aprobación.

Ventaja: identifica casos omitidos y valida la lógica de negocio.

Ejemplo: Tabla de Decisión (1/2)

Contexto: Sistema de cálculo de primas de seguro para automóviles.

- Prima base: **500 USD.**
- Reglas:
 1. Conductor < 25 años, masculino y no casado: recargo +1500 USD.
 2. Conductor casado o femenino: descuento -200 USD.
 3. Conductor entre 45 y 65 años: descuento adicional -100 USD.

Ejemplo: Tabla de Decisión (2/2)

```
1 def calcular_prima(edad, genero, casado):  
2     prima = 500  
3     if (edad < 25) and (genero == 'masculino') and (not casado):  
4         prima += 1500  
5     else:  
6         if casado or (genero == 'femenino'):  
7             prima -= 200  
8         if (edad > 45) and (edad < 65):  
9             prima -= 100  
10    return prima
```

Paso 1. Identificar condiciones y acciones

Condiciones (C):

- C1: Edad < 25
- C2: Género = Masculino
- C3: Casado = Sí
- C4: $45 \leq \text{Edad} \leq 65$

Acciones (A):

- A1: Recargo +1500
- A2: Descuento -200
- A3: Descuento -100

Paso 2. Construcción de la tabla de decisión

Reglas	C1 Edad<25	C2 M	C3 Casado	C4 45-65	A1 +1500	A2 -200	A3 -100
R1	S	S	N	N	X		
R2	N	-	S	N		X	
R3	N	N	N	N		X	
R4	N	-	-	S			X
R5	N	S	N	S			X
R6	N	-	S	S		X	X

Convenciones: S = Sí, N = No, - = No importa, X = acción aplicada.

Paso 3. Selección de casos de prueba

Caso	Edad	Género	Casado	Resultado esperado
T1	22	M	No	$\text{Prima} = 500 + 1500 = 2000$
T2	30	F	Sí	$\text{Prima} = 500 - 200 = 300$
T3	30	M	No	$\text{Prima} = 500 - 200 = 300$
T4	50	M	No	$\text{Prima} = 500 - 100 = 400$
T5	50	M	No	$\text{Prima} = 500 - 100 = 400$
T6	60	F	Sí	$\text{Prima} = 500 - 200 - 100 = 200$

Pruebas de Caja Blanca

Se diseñan con conocimiento de la estructura interna del software.

- **Objetivo:** Asegurar que cada parte del código se ejecute al menos una vez y siga el flujo lógico previsto.
- **Ventajas:** Detecta defectos en caminos poco usados, garantiza cobertura de condiciones.
- **Limitaciones:** Requiere esfuerzo de diseño y conocimientos de programación.

Métricas comunes: Cobertura de Sentencias, Cobertura de Decisiones, Cobertura de Condiciones.

Cobertura de Sentencias

Verifica que cada instrucción o línea de código se ejecute al menos una vez.

- **Pasos:** Identificar todas las sentencias, diseñar pruebas que las recorran.
- **Ejemplo:** En un if/else, provocar la ejecución de ambas ramas.

Ventaja: asegura la ejecución de todo el código. **Limitación:** no cubre todas las combinaciones lógicas.

Ejemplo - Cobertura de Sentencias (1/4)

```
1 def calcular_prima(edad, genero, casado):  
2     prima = 500  
3     if (edad < 25) and (genero == 'masculino') and (not casado):  
4         prima += 1500  
5     else:  
6         if casado or (genero == 'femenino'):  
7             prima -= 200  
8             if (edad > 45) and (edad < 65):  
9                 prima -= 100  
10    return prima
```

Ejemplo - Cobertura de Sentencias (2/4)

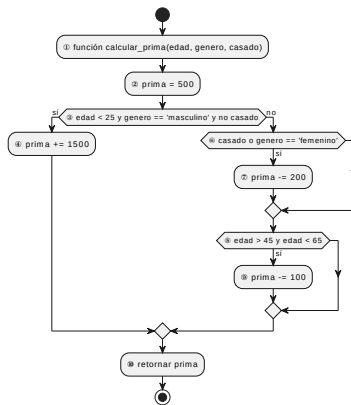


Figura 10: Diagrama de Flujo

Ejemplo - Cobertura de Sentencias (3/4)

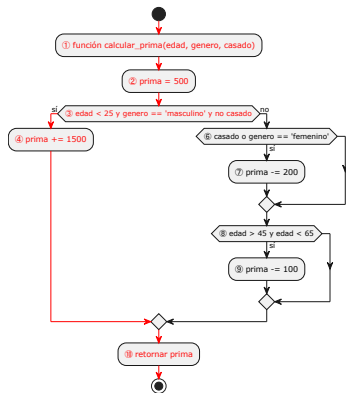


Figura 11: Diagrama de Flujo

Caso de Prueba #1:

TC1 : `calcular_prima(20, "masculino", False)`

Ejemplo - Cobertura de Sentencias (4/4)

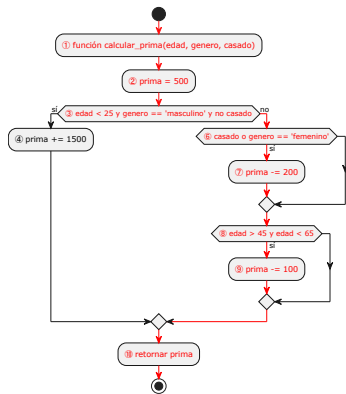


Figura 12: Diagrama de Flujo

Caso de Prueba #2:

TC2 : `calcular_prima(50, 'femenino', True)`

Cobertura de Decisiones

Comprueba que cada posible resultado (verdadero/falso) de las decisiones se ejecute al menos una vez.

- **Pasos:** Identificar decisiones (if, switch), diseñar pruebas para ambos resultados.
- **Ejemplo:** Para if ($x > 10$), casos con $x > 10$ y $x \leq 10$.

Ventaja: detecta defectos en ramas no cubiertas por la cobertura de sentencias.

Ejemplo - Cobertura de Decisiones (1/3)

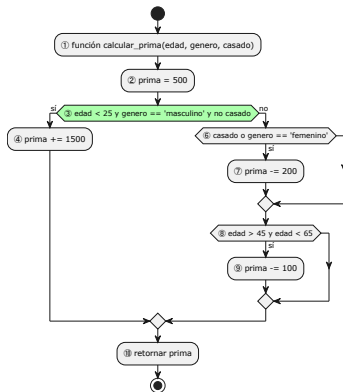


Figura 13: Enter Caption

Casos de Prueba:

TC1 : `calcular_prima(20, "masculino", False)`

TC2 : `calcular_prima(50, 'femenino', True)`

Ejemplo - Cobertura de Decisiones (2/3)

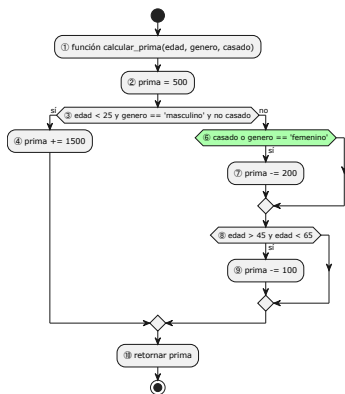


Figura 14: Enter Caption

Casos de Prueba:

TC3 : `calcular_prima(30,' masculino', False)`

TC4 : `calcular_prima(30,' masculino', True)`

Ejemplo - Cobertura de Decisiones (3/3)

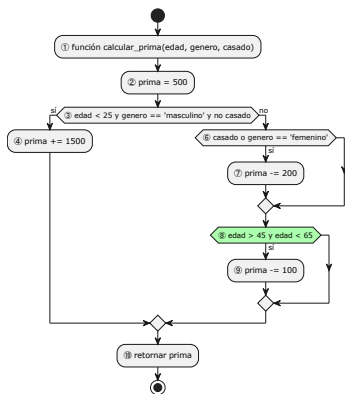


Figura 15: Enter Caption

Casos de Prueba:

TC5 : `calcular_prima(44, "masculino", False)`

TC6 : `calcular_prima(60, "femenino", False)`

Cobertura de Condiciones

Garantiza que cada condición elemental dentro de una expresión booleana sea evaluada como verdadera y falsa.

- **Pasos:** Descomponer cada decisión en condiciones simples, diseñar pruebas que cubran todos los valores.
- **Ejemplo:** En `if (A && B)`, probar A y B en verdadero y falso de manera controlada.

Ventaja: cobertura más fina que la de decisiones.

Limitación: requiere más casos de prueba para lograr cobertura completa.

Casos de Prueba para Cobertura de Condiciones

Caso	edad	género	casado	C1: edad<25	C2: masc	C3: not casado	C4: casado	C5: fem	C6: edad>45	C7: edad<65
1	20	masculino	False	True	True	True	False	False	False	True
2	30	femenino	True	False	False	False	True	True	False	True
3	50	masculino	False	False	True	True	False	False	True	True
4	70	femenino	False	False	False	True	False	True	True	False
5	45	masculino	True	False	True	False	True	False	False	True
6	65	femenino	False	False	False	True	False	True	True	False

Tabla 1: Casos de prueba y evaluación de condiciones atómicas

Casos donde cada condición se evalúa como verdadera

Condición	Casos donde se evalúa como True
C1: edad < 25	Caso 1
C2: género == 'masculino'	Casos 1, 3, 5
C3: not casado	Casos 1, 3, 4, 6
C4: casado	Casos 2, 5
C5: género == 'femenino'	Casos 2, 4, 6
C6: edad > 45	Casos 3, 4, 6
C7: edad < 65	Casos 1, 2, 3, 5

Tabla 2: Casos donde cada condición se evalúa como verdadera

Casos donde cada condición se evalúa como falsa

Condición	Casos donde se evalúa como False
C1: edad < 25	Casos 2, 3, 4, 5, 6
C2: género == 'masculino'	Casos 2, 4, 6
C3: not casado	Casos 2, 5
C4: casado	Casos 1, 3, 4, 6
C5: género == 'femenino'	Casos 1, 3, 5
C6: edad > 45	Casos 1, 2, 5
C7: edad < 65	Casos 4, 6

Tabla 3: Casos donde cada condición se evalúa como falsa

5. Referencias

Referencias (1/1)

Albeiro Espinosa Bedoya, Ph.D. Ms.C.

Profesor Asociado

Email: aespinos@unal.edu.co

Instagram: [@albeiroespinosabedoya](https://www.instagram.com/albeiroespinosabedoya)

**Departamento de Ciencias de la Computación y de la
Decisión, Facultad de Minas**

Dirección: Av. 80 #65-223, Bloque M8A, Of. 204

Teléfono: +57 604 4255384
